Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»
(МФЮА)

КАФЕДРА «ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

по дисциплине: «Разработка программных модулей»

на тему: «Разработка модуля управления каталогом товаров для интернет-магазина»

Специальность: 09.02.07
Информационные системы и программирование
Профиль: Программист

Автор работы: Чемоданов Никита Сергеевич
Курс: 4 Группа: 23ИПо8381 (ИПо-41)
Руководитель работы: Колпащиков Кирилл Александрович
Подписи и даты: _____

Киров — 2026

СОДЕРЖАНИЕ

Введение.....3
1 Теоретические основы и анализ управления товарным каталогом.....5
1.1 Особенности ведения каталога товаров интернет-магазина.....5
1.2 Подходы к автоматизации и требования к модулю.....7
1.3 Обоснование выбора технологий.....10
2 Проектирование модуля управления каталогом.....13
2.1 Пользовательские сценарии и функциональная модель.....13
2.2 Проектирование базы данных и ограничений целостности.....16
2.3 Архитектура приложения и проектирование интерфейса.....20
3 Разработка и тестирование модуля.....23
3.1 Реализация операций с товарами и категориями.....23
3.2 Реализация поиска, фильтрации и веб-интерфейса.....25
3.3 Тестирование, анализ результатов и ограничения решения.....27
Заключение.....33
Список использованных источников.....35
Приложение А. Экраны и результаты демонстрации.....38

ВВЕДЕНИЕ

Менеджер каталога добавляет товары, меняет цены и остатки, распределяет позиции по категориям. При такой работе возникают ошибки: повторяется артикул, вводится отрицательное количество или выбирается удалённая категория. Программа должна отклонять эти данные и объяснять причину отказа. В курсовой работе разработан модуль, который выполняет перечисленные операции через веб-формы и JSON API, применяя общие правила проверки.

Работа выполняется по дисциплине «Разработка программных модулей» для специальности 09.02.07 «Информационные системы и программирование». Выданное задание определяет операции добавления, просмотра, изменения и удаления товаров, поиск, фильтрацию, сортировку и хранение данных. В качестве документационной основы в задании назван ГОСТ 19.201–78 [1]. Структура и базовое оформление записки приняты по предоставленным методическим указаниям МФЮА [2]. Объём проекта ограничен управлением каталогом: заказы, расчёты с покупателями и доставка не рассматриваются.

При проектировании использованы материалы о типах и классах Python [3], разделении прикладных операций и обработчиков запросов [5], работе с небольшими воспроизводимыми примерами [7]. Для выбора проверок рассмотрены функциональные тесты пользовательских сценариев и тесты отдельных компонентов [6; 8]. В проекте эти виды проверок разделены на тесты базы данных, API и HTML-страниц.

Цель работы — разработать и проверить модуль управления каталогом товаров интернет-магазина, обеспечивающий ведение товаров и категорий, поиск и совместное применение условий отбора. Для достижения цели поставлены задачи: проанализировать предметную область и требования; обосновать технологии; спроектировать сущности, ограничения и интерфейс; реализовать операции и обработку ошибок; подготовить демонстрационные

данные; проверить положительные и отрицательные сценарии; оценить результаты и ограничения решения.

Объект исследования — процесс ведения товарного каталога интернет-магазина. Предмет исследования — структуры данных, алгоритмы обработки запросов и программные средства автоматизации хранения, изменения и поиска сведений о товарах и категориях. В качестве учебного ассортимента используются канцелярские товары. Сведения не относятся к обследованному предприятию; цены и остатки в демонстрационном наборе условные.

В работе применены анализ документации, сравнение вариантов хранения, моделирование данных и пользовательских сценариев, программирование и функциональное тестирование. Результат проверяется через действия с каталогом: сохраняется ли товар с допустимыми данными, отклоняется ли повторный артикул, остаются ли записи после перезапуска. Экономический эффект и производительность под промышленной нагрузкой не измерялись.

Результат работы — локальное приложение, исходный код, инструкция запуска и набор тестов. Через веб-страницы и JSON API можно вести один и тот же каталог. Первая глава записки описывает требования и выбор технологий, вторая — модель данных и устройство приложения, третья — реализацию и результаты проверок. В приложении приведены снимки работающей программы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА

Товарный каталог в рамках проекта представляет собой совокупность записей об ассортименте. Каждая запись содержит название, артикул, категорию, цену, описание и текущее количество. Название служит для распознавания и поиска позиции, но не обязано быть уникальным. Артикул позволяет однозначно отличить товар от других позиций. Например, два варианта ручки могут иметь близкие названия, однако различаться цветом, толщиной линии и артикулом. Поэтому запрещать все совпадения названий было бы излишним, а разрешать повторный артикул — противоречить принятому правилу идентификации.

Идентификатор записи создаётся базой и используется в связях и адресах карточек. Артикул задаёт менеджер. Если он исправляет артикул товара, идентификатор остаётся прежним: меняется существующая запись. Уникальность нового артикула проверяется среди остальных товаров.

Категория объединяет товары по назначению. Её название и описание хранятся отдельно, а товар содержит ссылку на соответствующую запись. Если категория переименована, товары продолжают относиться к ней по прежнему идентификатору. Название категории не требуется вручную исправлять у каждой позиции. Для первой версии выбрана одна категория на товар. Это упрощает форму и условия отбора; иерархическая классификация потребовала бы дополнительных правил перемещения и удаления узлов.

Цена и количество отражают разные свойства. Нулевая цена допускается, отрицательная отклоняется. Остаток определяется как целое число единиц, не меньшее нуля. Товар с нулевым остатком сохраняется в общем каталоге, но не входит в выдачу при фильтре наличия. В отличие от текущего остатка журнал

складских операций должен описывать причины изменения количества: приход, списание, резервирование. В принятой модели такой журнал отсутствует, поэтому изменение числа в карточке не является полноценным складским учётом.

Представление предметных сущностей в программе связано с выбором типов и операций. В учебнике Think Python классы рассматриваются как способ объединить пользовательский тип и связанное с ним поведение, а словари — как отображение ключей в значения [3, гл. 10, 14]. В проекте сущности представлены моделями, а допустимые данные запроса — отдельными схемами. Это позволяет различать хранимую запись, введённые значения и данные, подготовленные для ответа, даже если часть их полей совпадает.

Основной пользователь модуля — менеджер каталога. Он создаёт категории и товары, находит нужные позиции, исправляет или удаляет их. После успешного сохранения открывается карточка либо список. При ошибке форма сохраняет введённые значения и показывает причину отказа, чтобы менеджеру не приходилось заполнять все поля заново.

В рамках каталога запись описывает товарную позицию, а не отдельную физическую единицу. Десять одинаковых ручек представлены одним товаром с количеством 10. Если отличаются признаки, по которым менеджер должен вести отдельный остаток, создаются разные позиции с собственными артикулами. Так, синие и чёрные ручки могут иметь одинаковую цену и находиться в одной категории, но изменяются независимо. Такое толкование определяет, какие данные вводятся в карточку и что означает количество.

Проверка уникальности названия товара дала бы нежелательный запрет на позиции разных размеров или упаковок с одинаковым кратким обозначением. Поэтому название используется для поиска и отображения, а уникальность закреплена за артикулом. Для категорий принято другое правило: названия «Бумага» и «БУМАГА» обозначают один раздел. Различие правил связано с назначением сущностей. Категория организует справочник, а товар

описывает конкретную позицию ассортимента; переносить одинаковые ограничения на оба объекта было бы ошибкой.

Сведения в карточке разделены на обязательные и дополнительные. Без названия менеджер не распознает позицию, без артикула не проверит её уникальность, без категории не сможет применить соответствующий отбор. Описание уточняет характеристики и может отсутствовать. Дата создания и дата изменения заполняются программой, поскольку их ручной ввод не нужен для операций каталога. Зафиксированные правила позволяют определить допустимое состояние записи до разработки формы и не зависеть от её внешнего оформления.

1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ

Для учебной задачи рассмотрены три подхода: ведение перечня в таблице, настройка готовой системы магазина и разработка самостоятельного модуля. Сравнение выполнено по требованиям проекта. Испытание конкретных коммерческих продуктов, расчёт стоимости внедрения и измерение времени работы менеджера не проводились. Поэтому выводы характеризуют соответствие выбранной задаче, а не универсальное преимущество одной технологии.

Табличный перечень позволяет описать строки товаров и столбцы свойств. Однако связанные категории, уникальность артикула и поведение при ошибке необходимо организовать отдельными правилами. Таблица полезна как исходное представление данных, но не показывает реализацию обработки HTTP-запросов и серверной проверки. Готовая система требует выбора продукта, настройки и, при необходимости, расширения. Этот путь решает задачу внедрения системы, тогда как цель курсовой — спроектировать и объяснить собственный программный модуль на Python.

Для курсовой выбран самостоятельный веб-модуль: в нём можно реализовать и объяснить обработку запросов, проверку данных и запись в базу. Сравнение подходов приведено в таблице 1. Работа с небольшим запускаемым

проектом также позволяет разбирать отдельные операции на воспроизводимых примерах [7].

Таблица 1 — Проектное сравнение подходов

Подход	**Что организуется**	**Соответствие задаче**
Таблица товаров	Столбцы свойств, правила ввода и связи нужно настроить	Подходит для исходного перечня; HTTP-модуль не реализует
Готовая система	Выбор продукта, настройка или расширение	Возможный прикладной путь; самостоятельная разработка не показана
Отдельный модуль	Собственные схема, операции, формы и проверки	Выбран для изучения Python и FastAPI

Функциональные требования включают создание, просмотр, изменение и удаление товаров и категорий, подробную карточку товара, поиск по названию, фильтрацию категории и цены, признак наличия, сортировку и постраничный вывод. Условия отбора должны сочетаться. Поиск не зависит от регистра русского текста; ценовые границы включают равные значения. Страница списка содержит не более десяти товаров, а ссылки переходов сохраняют выбранные условия.

Требования к корректности задают ограничения: товар связан с существующей категорией; артикул и нормализованное имя категории уникальны; цена и остаток неотрицательны; остаток целочисленный. Удаление занятой категории отклоняется. После остановки и запуска обычного сервера данные сохраняются. Ошибка запроса не должна приводить к частичному изменению другой записи. Эти требования проверяются наблюдаемым результатом и состоянием базы, а не только наличием элементов формы.

Для каждой операции проверяется успешный запрос и причины отказа. При создании товара это повторный артикул, неправильная цена и отсутствующая категория; при редактировании дополнительно проверяется сохранность прежних данных после ошибки. Вводная глава The Fuzzing Book объясняет проверку через сопоставление результата с ожидаемым поведением и отмечает ограниченность отдельных тестовых примеров [10].

Требования рассматриваются на трёх уровнях. Пользовательский уровень описывает действие менеджера: добавить товар, исправить цену, найти позиции определённой категории. Уровень программного интерфейса задаёт состав запроса, формат цены и код ответа. Уровень хранения определяет уникальность, типы чисел и обязательность связей. Например, требование «не допускать повторного артикула» выражается проверкой операции, ответом 409 и ограничением UNIQUE. Ни один отдельный элемент формы не подтверждает выполнение всего требования.

Для приёмки операции задаются исходное состояние, действие и ожидаемый результат. Перед проверкой удаления занятой категории в базе должны существовать категория и связанный товар. Затем выполняется запрос удаления. Правильный результат включает отказ и сохранение обеих записей. Если проверить только текст сообщения, можно пропустить ошибочное изменение базы. По той же причине проверка редактирования должна читать запись после сохранения, а проверка неуспешного редактирования — сравнивать её с прежними значениями.

Требования к удобству в проекте выражены через конкретные действия. Менеджер должен видеть, какое поле не принято, и иметь возможность исправить его без повторного заполнения формы. При удалении он сначала видит название и артикул объекта. При переходе между страницами сохраняются фильтры. Эти свойства проверяются через ответы HTML и браузерные сценарии. Оценка удовлетворённости пользователей и сравнение времени выполнения операций с другими системами не проводились, поэтому такие показатели не используются как результат работы.

1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Основной язык разработки — Python 3.12. Он позволяет описать входные данные, модели и операции в одном проекте. Используемые пакеты закреплены в файлах зависимостей, а виртуальное окружение отделяет их от системных библиотек. Для воспроизведения результатов важны не только исходники, но и сведения об окружении и последовательности запуска; соответствующий подход к происхождению результатов рассматривается в Research Software Engineering with Python [4].

FastAPI выбран для обработки HTTP-запросов и предоставления JSON API. Пользовательские страницы формируются сервером с Jinja2. Документация FastAPI описывает передачу объекта запроса, имени шаблона и контекста в TemplateResponse, а также подключение статических файлов [13]. Такая организация подходит для небольшого интерфейса менеджера: после отправки формы сервер проверяет данные и возвращает страницу либо перенаправление. Основные операции не зависят от JavaScript.

Для работы с данными применяется SQLAlchemy, для входных схем — Pydantic. Сессия SQLAlchemy связывает загруженные объекты с транзакцией и предоставляет явные commit и rollback [14]. Строгий режим Pydantic ограничивает неявные преобразования типов [16]. В API количество и идентификатор категории принимаются целыми JSON-числами. HTML передаёт строки, поэтому адаптер формы отдельно разбирает допустимую целочисленную запись, затем применяет ту же схему.

СУБД выбрана с учётом локальной демонстрации. Согласно документации Python, SQLite хранит данные в файле и не требует отдельного серверного процесса [11]. В задании среди источников названа PostgreSQL. Её документация использована для анализа таблиц, ключей и ограничений [12]. При этом цель задания не закрепляет конкретную СУБД, а фактически реализованная версия использует SQLite. Это решение уменьшает число

действий при установке на Mac и позволяет создавать отдельные файлы для тестов.

Выбор SQLite не означает доказанной пригодности к произвольной нагрузке. Перенос на PostgreSQL не выполнялся. Несмотря на общий ORM, SQLite-специфичные проверки typeof и GLOB, параметры подключения и PRAGMA потребовали бы пересмотра. При переносе нужно проверить схему, подключение и поведение запросов. Одновременная работа нескольких менеджеров требует отдельного тестирования.

Тесты базы, API и HTML выполняются через pytest и тестовый HTTP-клиент, действия в браузере — через Playwright [17; 19]. Для локального запуска используется Uvicorn: ему передаются модуль приложения, адрес и порт [20].

Выбранный способ формирования страниц не требует отдельного клиентского приложения и сборщика JavaScript. Маршрут получает запрос, вызывает операцию каталога и передаёт результат шаблону. Это позволяет выполнять основные действия при отключённом JavaScript и проверять страницы тем же тестовым клиентом, что и API. При этом HTML остаётся отдельным представлением: ошибки формы и данные JSON-запроса обрабатываются разными адаптерами. Общими остаются правила записи и поиска, а не весь код ответа.

Синхронный доступ к SQLite соответствует выбранному режиму локальной демонстрации. Для каждого запроса используется отдельная сессия SQLAlchemy; одна глобальная сессия для всех действий не создаётся. Из этого решения не следует вывод о допустимом числе одновременных пользователей. Если область применения расширится, потребуется отдельно проверить конкурирующую запись, блокировки и восстановление после отказов. Само наличие веб-сервера не является результатом такой проверки, а замена синхронных функций асинхронными не устраняет ограничения модели данных.

Версии приложения и инструментов тестирования закреплены в requirements.txt и requirements-dev.txt. Установка в виртуальное окружение

отделяет их от системных пакетов, а команда pip check выявляет несовместимые требования установленных библиотек. Проверка зависимостей дополняет запуск тестов, но не заменяет его: согласованные версии могут содержать ошибку приложения. Для повторения демонстрации нужны исходники, Python 3.12 и последовательность запуска из README. Рабочая база создаётся локально, поэтому её отсутствие в Git не означает отсутствие структуры данных в проекте.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ

Пользовательские сценарии начинаются с формирования структуры каталога. Менеджер создаёт категорию, вводя название и необязательное описание. Если эквивалентное нормализованное название уже существует, сервер отклоняет запись и возвращает форму. После успешного сохранения категория доступна для выбора при создании товара. При пустом справочнике форма товара предлагает сначала создать категорию, поскольку ссылка на неё обязательна.

Создание товара включает название, артикул, категорию, цену и остаток. Описание допускается не заполнять. После проверки сервер сохраняет запись и открывает карточку с результатом. При редактировании форма содержит текущие значения; сохраняется тот же идентификатор. Если новый артикул занят, изменённый товар не должен частично получить другую цену или название. Соответствующие проверки выполняются до изменения объекта и повторяются ограничением уникальности при записи.

Поиск сочетается с категорией, нижней и верхней границами цены, наличием и сортировкой. Отключённое условие не ограничивает выдачу. Признак наличия означает количество больше нуля, а не положительную цену. При переходе на следующую страницу сохраняются все условия. Пустая выдача отличается от отсутствия данных в новом каталоге: в первом случае предлагается изменить отбор, во втором — создать первые записи. Связь требований и признаков корректности приведена в таблице 2.

Таблица 2 — Связь требований с проверками

Требование	**Реализация**	**Признак корректности**
CRUD товара	Форма, карточка, API	Изменения видны после чтения; ID сохраняется
Категории	Справочник и связь	Повторное имя и удаление занятой категории отклоняются
Поиск	Нормализованное имя	Русский регистр не меняет результат
Фильтры	Условия общего запроса	Категория, цена и наличие применяются вместе
Пагинация	LIMIT 10 и OFFSET	120 страниц для 1200 товаров
Неправильный ввод	Схемы и форма	Статус 422; заполненные значения сохранены
Данные после старта	Файловая SQLite	Повторное открытие не удаляет записи

Удаление товара разделено на открытие подтверждения и изменение базы. GET показывает сведения и выбор действия; только POST выполняет удаление. Отмена возвращает карточку. Удаление занятой категории вместо подтверждения показывает причину запрета и ссылку на её товары. Сервер проверяет это правило повторно при POST, поскольку состав категории мог измениться после открытия страницы.

Неправильные данные возвращают статус 422, отсутствие объекта — 404, конфликт уникальности или связи — 409. API передаёт ошибку в JSON, веб-интерфейс показывает HTML-страницу с сообщением. Успешное создание через API возвращает 201, удаление — 204 без тела. После сохранения HTML-формы сервер отвечает перенаправлением 303, и браузер открывает результат запросом GET.

Сценарии связываются с проверками ещё на этапе проектирования. В работе Х. Персиваля функциональный тест описывает поведение с точки

зрения пользователя, а более узкие проверки помогают локализовать отказ [6, гл. 4]. В проекте применены оба масштаба: запросы к API и HTML проверяют наблюдаемый результат, проверки базы — ограничения хранения, а браузерный прогон — выполнение последовательности действий на страницах.

Редактирование товара можно описать как последовательность состояний. Сначала GET загружает существующую запись и заполняет форму. Затем менеджер отправляет изменённые поля. До успешной фиксации экран с новым вводом ещё не означает изменения базы. При ошибке сервер повторно показывает форму со статусом 422 или 409; при успехе выполняет перенаправление на карточку. Благодаря этому различаются введённые значения, сохранённая запись и представление результата, хотя пользователь видит их в одном интерфейсе.

Подтверждение удаления относится к конкретному идентификатору, а не к текущему положению строки в списке. Между открытием подтверждения и отправкой POST запись может измениться или исчезнуть. Поэтому изменяющий маршрут заново получает объект по идентификатору и выполняет проверку ограничений. Отдельная страница подтверждения снижает риск случайного нажатия, но не заменяет серверную проверку. Открытие её адреса, переход по ссылке и обновление страницы должны оставаться операциями чтения.

Отсутствующий объект и неправильный запрос требуют разной реакции. Существующий по формату идентификатор, для которого нет записи, приводит к 404. Отрицательное или слишком большое число не проходит проверку параметра и даёт 422. Аналогично допустимый диапазон цены с пустой выдачей возвращает успешный список, а диапазон с нижней границей выше верхней отклоняется. Такие различия помогают менеджеру понять причину результата и позволяют тестам проверять поведение без разбора произвольного текста страницы.

JSON API предоставляет те же операции без браузерных форм. Например, после создания товара ответ содержит его идентификатор, название категории и

цену в согласованном формате. Дальнейшее чтение по этому идентификатору должно вернуть сохранённые значения. Это используется в автоматических сценариях, где результат одной операции становится входными данными следующей. В проекте такой способ работы нужен для проверки программного контракта; подключение внешней учётной системы или действующего магазина не выполнялось.

2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Модель включает сущности «Категория» и «Товар» со связью один ко многим. Категория имеет первичный ключ id, название, нормализованное название и описание. Товар содержит id, название, его нормализованную копию, артикул, описание, category_id, цену в копейках, количество и даты создания и изменения. Каждая товарная запись ссылается на одну существующую категорию. Логическая структура показана на рисунке 1.

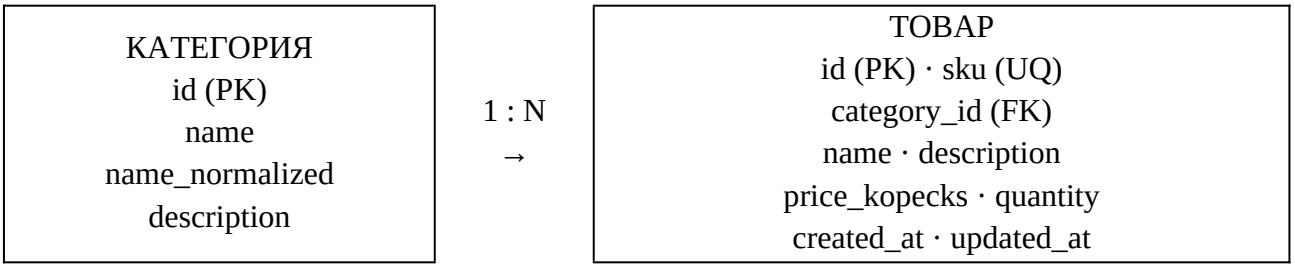


Рисунок 1 — Логическая структура данных каталога

Источник: составлено по модели и исходному коду проекта.

Разделение справочника и товаров предотвращает повторение пользовательского названия категории в каждой товарной строке. Это делает переименование одной операцией над справочником. Принципы первичного и внешнего ключей и ограничения уникальности рассмотрены в документации PostgreSQL [12]. В проекте они реализованы средствами SQLite и SQLAlchemy. Для внешнего ключа задан RESTRICT: наличие связанных товаров блокирует удаление категории.

Цена хранится целым числом копеек. Строка рублей проверяется по допустимому формату и преобразуется через Decimal. Принимаются точка и

запятая, не более двух знаков после разделителя; лишняя точность отклоняется без скрытого округления. Документация decimal объясняет возможность точного представления десятичных значений в отличие от ряда значений float [11]. При выводе целые копейки снова преобразуются в строку рублей с двумя дробными знаками.

Количество ограничено неотрицательным целым в диапазоне SQLite. Входные схемы отвергают логические и дробные значения. На уровне хранения CHECK проверяет целочисленный вид записанного значения и неотрицательность. Уникальность артикула и нормализованного имени категории обеспечивается индексами. Предварительная проверка приложения улучшает сообщение пользователю, но не заменяет ограничение базы при конкурирующих запросах.

Для поиска используется дополнительное нормализованное имя: NFKC, удаление крайних пробелов и casefold. Согласно документации unicodedata, NFKC сочетает совместимостное разложение с канонической композицией [11]. Исходное название сохраняется для отображения. Служебные поля заполняются моделями при записи через приложение. Здесь нормализация означает преобразование Unicode-строки, а не изменение реляционной схемы. Основные поля и правила сведены в таблицу 3.

Таблица 3 — Основные поля модели данных

Сущность и поле	**Представление**	**Правило**
Category.id	Целое	Первичный ключ
Category.name	Строка до 100 символов	Непустое; уникальна нормализованная форма
Category.description	Текст или NULL	Не более 2000 символов
Product.id	Целое	Первичный ключ
Product.name	Строка до 200 символов	Непустое; нормализованная копия для поиска
Product.sku	Строка до 64 символов	Уникальный; латинские буквы, цифры, дефис, подчёркивание
Product.category_id	Целое	Внешний ключ; категория существует; RESTRICT
Product.price_kopecks	Целые копейки	От 0 до 2^63 − 1; при вводе рубли строкой
Product.quantity	Целое количество	От 0 до 2^63 − 1
Product.description	Текст или NULL	Не более 2000 символов
Product.created_at, updated_at	Дата и время	Хранятся в UTC; обновление при изменении

Связь таблиц: categories.id → products.category_id, один ко многим. Удаление занятой категории запрещено. Нормализованные поля служебные и не входят в публичную схему API.

Пустое описание после удаления крайних пробелов сохраняется как NULL. Даты записываются в UTC без информации о часовом поясе в самом поле SQLite; API добавляет обозначение UTC, а браузер может показывать местное время. Поле остатка содержит текущее число единиц. Отдельных записей о приходе и списании в базе нет.

В таблицах разделены ограничения отдельного поля и ограничения связи. NOT NULL запрещает отсутствие обязательного значения, CHECK проверяет допустимое содержимое, UNIQUE запрещает повтор, а внешний ключ связывает товар с категорией. Эти проверки решают разные задачи. Например, положительный category_id ещё не подтверждает существование категории; для этого нужен внешний ключ. Корректный формат артикула также не гарантирует его уникальности. Поэтому проверка только длины строк и диапазона чисел была бы недостаточной.

Предел целого числа SQLite равен 2^63 − 1. Для цены он относится к копейкам, поэтому наибольшая принимаемая строка рублей — 92233720368547758.07. Следующее значение с окончанием .08 уже выходит за предел хранения. Это техническая граница представления, а не разумная цена товара. Проверка такой границы нужна, чтобы недопустимый ввод завершался ошибкой запроса, а не переполнением при записи. Для количества действует тот же предел целого, но преобразование из рублей к нему не применяется.

Хранение цены в копейках сохраняет точность сравнения границ: 49,90 рубля превращаются в 4990, а 50 рублей — в 5000. Фильтр может сравнивать целые значения без округления при каждом запросе. Преобразование выполняется до записи через Decimal; строка с тремя дробными знаками отклоняется. Внутреннее поле price_kopecks не принимается из публичного запроса. Клиент передаёт цену в рублях, а сервер сам получает представление хранения и возвращает цену с двумя знаками после разделителя.

Нормализованная копия названия поддерживается при присваивании через модель. Поэтому обычное переименование одновременно обновляет строку для поиска и сохраняет исходное написание для показа. Прямой SQL обходит Python-валидаторы и может нарушить соответствие этих полей, даже если ограничения длины выполнены. Штатные операции приложения записывают данные через модели; при будущем импорте из внешнего файла нужно будет использовать тот же путь или явно повторять подготовку значений. Само наличие служебного столбца не обеспечивает его актуальность.

2.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА

Компоненты организованы по назначению: database.py создаёт движок и сессии; models.py описывает хранимые сущности; schemas.py проверяет ввод; catalog.py реализует общие операции; маршруты API и страниц адаптируют запросы и ответы. Шаблоны и статические файлы отвечают за представление. Ответственность компонентов приведена в таблице 4, путь запроса — на рисунке 2.

Таблица 4 — Ответственность компонентов

Компонент	**Назначение**
database.py	Движок, внешний ключ, сессия на запрос
models.py / schemas.py	Таблицы, ограничения и проверка входных данных
catalog.py	Общие операции, транзакции, поиск и фильтры
routers/api.py	JSON-контракт и статусы HTTP
routers/pages.py	HTML-формы, сообщения и перенаправления
templates / static	Представление, оформление и местное время
demo_data.py / seed_demo.py	Воспроизводимые примеры, backup и загрузка

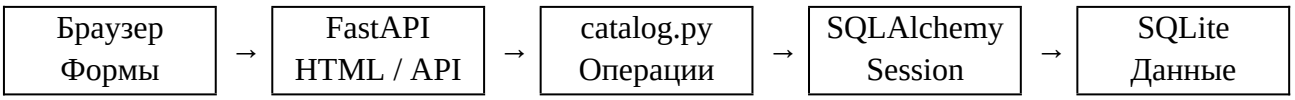


Рисунок 2 — Обработка запроса в приложении

Источник: составлено по модели и исходному коду проекта.

Общие операции вынесены из обработчиков запросов по принципу разделения прикладной логики и кода взаимодействия [5, гл. 4]. В модуле их реализуют функции catalog.py, работающие с сессией SQLAlchemy. Отдельный слой репозиториев, очередь событий и собственная реализация Unit of Work не используются.

HTML и API вызывают одинаковые функции каталога. Например, проверка занятости категории находится в общей операции, поэтому правило не зависит от того, каким интерфейсом воспользовался менеджер. Для каждого запроса выдаётся собственная сессия, успешная запись подтверждается явно, конфликт целостности сопровождается rollback. Идея атомарной операции обсуждается в книге Персиваля и Грегори [5, гл. 6], а конкретное поведение сессии определяется SQLAlchemy [14].

Интерфейс включает список товаров с фильтрами, карточку, общую форму создания и изменения, подтверждение удаления и страницы категорий. На широком экране меню расположено слева. На узком — сверху, поля формы выстраиваются последовательно. Таблица прокручивается в собственном контейнере. Для полей предусмотрены подписи, для ошибок — заметное сообщение с сохранённым вводом. Основные действия доступны без JavaScript; скрипт дополняет отображение дат местным временем.

Рабочие данные находятся в data/catalog.db. Этот файл исключён из Git; в репозитории хранятся код и генератор примеров. Обновление исходников не заменяет рабочую базу. Для тестов создаются отдельные SQLite-файлы. Выборка записей из базы по ключам рассматривается в учебных материалах Г. Уилсона [7].

Граница между слоями видна на примере ошибки уникальности. Функция каталога сообщает о конфликте через CatalogError и не выбирает HTML-шаблон. Маршрут страницы возвращает форму с введёнными значениями, а обработчик API — JSON с полем detail. Благодаря этому изменение оформления сообщения не требует менять алгоритм поиска дубликата. В обратную сторону шаблон не выполняет запросы записи и не принимает решение, можно ли удалить занятую категорию.

Зависимость get_db выдаёт сессию на время запроса и закрывает её после использования. Место фиксации находится в операции каталога. Такое разделение позволяет явно определить завершение записи и подменить доступ к базе в тестах. Если бы подключение создавалось внутри каждого шаблона или

использовался общий изменяемый объект сессии, изоляция тестовых запросов потребовала бы дополнительных обходных решений. В текущей структуре страницы и API получают сессию через один механизм зависимости FastAPI.

Для списка и формы нужны разные представления одних данных. В списке показываются краткое название, категория, цена и остаток; карточка содержит описание и даты. Форма дополнительно передаёт ограничения длины и допустимого ввода в HTML-атрибутах. Серверная схема остаётся обязательной, поскольку запрос можно отправить без браузера. Шаблоны отвечают за подписи и расположение элементов, а схема определяет, какие значения могут поступить в операцию каталога.

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ

Точкой входа служит app/main.py. При запуске lifespan создаёт отсутствующие таблицы. Путь к data/catalog.db вычисляется от папки проекта. Повторная инициализация сохраняет записи, но не выполняет миграцию существующей схемы. Локальный запуск выполняется Uvicorn с адресом 127.0.0.1 и портом 8000; назначение параметров соответствует документации сервера [20]. Локальная работа приложения на macOS дополнительно проверена студентом.

API содержит десять операций: получение списка, создание, чтение одной записи, изменение и удаление для каждой сущности. Создание товара сначала проверяет схему, наличие категории и уникальность артикула. После добавления объекта выполняется commit, затем обновление его состояния для ответа. Артикул приводится к верхнему регистру. При изменении проверяется существующая запись, новая категория и артикул, после чего заменяются редактируемые поля.

Проверка до присваивания важна для отказов редактирования. Например, попытка заменить артикул занятой строкой не должна одновременно изменить цену. Если конфликт всё же возникает при записи, обработчик IntegrityError откатывает транзакцию и возвращает 409. Документация SQLAlchemy требует учитывать состояние сессии после неуспешной записи [14]. В тестах после конфликтного запроса выполняется корректный, чтобы проверить сохранность данных и возможность продолжить работу.

Перед удалением категории общая функция проверяет, есть ли в ней товары. Дополнительно удаление занятой категории запрещено внешним ключом с RESTRICT. Удаление отдельного товара сохраняет категорию. Поле updated_at обновляется средствами модели при изменении записи. Маршруты

возвращают результат в JSON или HTML. Для PUT требуется полный набор обязательных редактируемых полей; частичное обновление не реализовано.

HTML-форма возвращает введённые значения при 422 и конфликте 409. После успеха браузер получает перенаправление 303 на карточку либо список. RFC 9110 определяет назначение 303 See Other как переход к ресурсу, представляющему результат исходного запроса [18, п. 15.4.4]. Следующий просмотр выполняется GET, поэтому обновление карточки не повторяет сохранение. Открытие страницы подтверждения также не удаляет объект.

Обычные формы передают строки, тогда как схема количества и категории ожидает строгие целые числа. Адаптер допускает только десятичную целочисленную запись подходящей длины, затем передаёт число Pydantic [16]. Строгость JSON-контракта при этом сохраняется. Проверка в браузере помогает пользователю, но сервер повторяет её, поскольку HTTP-запрос может поступить без формы.

Обработка цены начинается с удаления крайних пробелов и проверки строки регулярным выражением. Допускаются цифры и один десятичный разделитель с одной или двумя последующими цифрами. После замены запятой точкой создаётся Decimal, проверяется предел и вычисляется целое число копеек. Поэтому значения «49», «49.9» и «49,90» приводятся к согласованному формату, а отрицательная цена, экспоненциальная запись и NaN отклоняются. Проверка формата выполняется раньше обращения к базе.

Строгая проверка целых полей исключает случай, когда логическое true становится количеством 1. Для JSON поля quantity и category_id должны быть целыми числами, а не строками или числами с дробной записью. HTML-адаптер преобразует только строку из 1–19 десятичных цифр; окончательный диапазон затем проверяет Pydantic. Копия словаря нужна, чтобы подготовка чисел для схемы не уничтожала исходные значения, возвращаемые в форму после ошибки.

Схемы входа запрещают лишние поля. Клиент не может задать id, name_normalized или внутреннюю цену в копейках вместе с публичными

полями товара. Это проверяется до изменения модели. При PUT обязательные поля передаются полностью, поэтому запрос только с новым названием отклоняется. Внутри операции редактирования текущий идентификатор исключается из поиска дубликата артикула: сохранение своего прежнего артикула допустимо, совпадение с другой записью — конфликт.

Функция _commit распознаёт известные нарушения ограничений SQLite и преобразует их в ошибки каталога. После IntegrityError сначала выполняется rollback. Если сообщение относится к повторному имени категории, артикулу или нарушенной связи, клиент получает предусмотренный ответ 409. Неизвестное нарушение повторно возбуждается как исключение. Такой порядок не скрывает любую внутреннюю ошибку под сообщением о дубликате и позволяет при отладке отличить ожидаемый отказ пользователя от дефекта реализации.

3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА

Отбор выполняет общая функция list_products. Поиск сравнивает нормализованную строку с нормализованным названием товара. Символы процента и подчёркивания экранируются как буквальный ввод для LIKE. К запросу добавляются выбранные ограничения категории, нижней и верхней цены и количества больше нуля. До выборки проверяется корректность диапазона; минимальная цена не может превышать максимальную.

Сортировка выбирается из фиксированного перечня: название по возрастанию и убыванию, цена по возрастанию и убыванию, недавнее изменение. При равных значениях используется идентификатор, чтобы порядок оставался однозначным. В list_products каждому варианту сортировки соответствует выражение SQLAlchemy; вторым условием задан Product.id. Тесты проверяют порядок при равных значениях основного поля.

Пагинация использует два результата: общее количество подходящих записей и ограниченную страницу данных. Размер страницы равен десяти, смещение определяется номером страницы. Форма фильтрации использует

GET, поэтому параметры остаются в адресе. Ссылки переходов сохраняют эти параметры и изменяют только страницу. Ошибочный запрос получает понятный HTML; пустая выдача остаётся успешным результатом, если условия допустимы.

Jinja2 использует общий базовый шаблон с навигацией и отдельные шаблоны списка, карточки, форм и подтверждений. Пользовательские значения передаются через контекст, как описано в документации FastAPI [13]. Включённое окружением шаблонов автоматическое экранирование обрабатывает HTML-символы; пользовательский текст не помечается safe. Документация Jinja описывает исключения из экранирования и зависимость поведения от настройки [15]. Тесты с введёнными тегами проверяют вывод текста без исполнения разметки.

Для демонстрации большого списка подготовлены 1200 синтетических товаров в 12 категориях. На каждую приходится 100 записей. Отличаются типы, варианты, цены и остатки; 133 товара имеют нулевое количество. Цена находится в диапазоне 19,00–346,10 рубля. Генератор не описывает прайс-лист действующего поставщика. Записи помечены как демонстрационные и имеют стабильные артикулы.

Загрузчик создаёт SQLite backup перед записью в существующую базу. Используется метод backup, описанный в документации sqlite3 [11], а не простое копирование открытого файла. Затем недостающее добавляется одной транзакцией. Имеющийся артикул пропускается, описание существующей категории сохраняется. Поэтому повторный запуск не отменяет пользовательскую правку цены и не создаёт дубль с тем же артикулом. Воспроизводимость набора позволяет проверять те же условия на другом компьютере.

Составной отбор строится как пересечение условий. Например, для запроса «РУЧКА», выбранной категории, диапазона 49,90–100,00 рубля и включённого наличия товар должен удовлетворять всем четырём ограничениям. В тестовом наборе синяя ручка за 49,90 и гелевая за 100,00

попадают в результат, а красная за 99,00 с нулевым остатком исключается. Пример одновременно проверяет регистр поиска, включение граничных цен и независимость наличия от цены.

Для подсчёта и выборки страницы используется один список условий. Сначала считается общее число подходящих строк, затем выполняется запрос с LIMIT 10 и OFFSET. Для первой страницы смещение равно нулю, для второй — десяти. При 13 результатах страницы содержат 10 и 3 записи. Число страниц вычисляется с округлением вверх; даже при пустом результате интерфейс сохраняет первую страницу. Если запрошен номер за пределами выдачи, данные не подменяются результатами другой страницы.

Категория загружается вместе с товаром через joinedload, поскольку в строке списка требуется её название. Это позволяет получить связанные данные в запросе выборки вместо отдельной загрузки категории при каждом обращении к объекту. Из такого устройства запроса не выводится измеренное ускорение: сравнение времени и числа запросов под нагрузкой не проводилось. Здесь решение объясняется тем, какие поля нужны представлению, и тем, что шаблон получает уже связанные объекты.

Ссылки пагинации формируются через urlencode из действующих параметров с заменой номера страницы. Это сохраняет поисковую строку, категорию и границы цены при переходе. Новая отправка формы фильтров не переносит прежний номер страницы, поэтому поиск начинается заново. Параметры удобно воспроизвести из адреса, однако адрес не является снимком данных: если каталог изменится, тот же запрос может вернуть другой состав позиций. Устойчивый порядок при равных значениях уменьшает неоднозначность выдачи, но не фиксирует состояние базы между запросами.

3.3 ТЕСТИРОВАНИЕ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОГРАНИЧЕНИЯ РЕШЕНИЯ

Проверка выполнена 21.10.2026 в облачной Linux-среде на Python 3.12. Автоматические тесты получают отдельные временные SQLite-файлы.

Фикстуры создают движок, схему и сессию, а тестовый клиент подменяет зависимость доступа к базе. Такой способ организации согласуется с механизмом фикстур pytest [17]. Рабочий каталог не используется для записи тестовых примеров. Его демонстрационное наполнение выполнено отдельно загрузчиком после резервного копирования.

Всего пройдено 154 теста: 35 проверок базы, 81 API, 31 HTML и 7 загрузчика. Проверяются операции, уникальность, типы и границы чисел, обязательность связи, поиск русского текста, сочетания фильтров, сортировка, пагинация и отказ без частичного изменения. Для загрузчика дополнительно проверены повторное применение, сохранение собственных записей, откат при искусственном сбое и резервная копия с зафиксированными данными WAL.

При запуске получено одно предупреждение Starlette об устаревающем использовании HTTPX в TestClient. Все тесты завершились успешно. Набор можно повторно запускать после изменения кода для проверки перечисленных сценариев; такую роль автоматических тестов рассматривает Software Engineering at Google [8, гл. 11].

Дополнительно выполнены 14 сценариев в Chromium 154.0.8037.57 при размерах экрана 1440 × 1000 и 390 × 844. Через Playwright проверены формы, сообщение о неправильной цене с сохранением ввода, совместный отбор, пагинация, защита категории, отмена и подтверждение удаления. Также проверена форма при отключённом JavaScript. Использован синхронный интерфейс автоматизации, соответствующий документации Playwright для Python [19].

Браузерный прогон выявил расширение мобильной страницы скрытой подписью таблицы. Ошибка воспроизведена на узком экране; позиционирование подписи ограничено прокручиваемым контейнером. Повторная проверка сравнила ширину документа с шириной окна. Проверка выполнена по схеме воспроизведения и локализации отказа, описанной в The Debugging Book [9]. Также исправлено выравнивание последнего столбца и добавлены проверки слишком больших идентификаторов адресов.

На этом наборе в Chromium выполнены ещё шесть сценариев большого каталога: количество записей API, переход по страницам, последняя страница, совместные фильтры, счётчики категорий и мобильная ширина. Список из 1200 товаров образует 120 страниц; в каждой категории отображается 100 товаров. Итоги сведены в таблицу 5. integrity_check вернул ok, foreign_key_check — отсутствие ошибок; повторная загрузка добавила ноль записей.

Таблица 5 — Фактические результаты проверки

Набор	**Количество**	**Результат**
База данных	35 тестов	Пройдены
JSON API	81 тест	Пройдены
HTML-страницы	31 тест	Пройдены
Загрузчик примеров	7 тестов	Пройдены
Браузер, основные сценарии	14 сценариев	Пройдены в Chromium
Браузер, 1200 товаров	6 сценариев	Пройдены в Chromium
SQLite большого набора	integrity_check, foreign_key_check	ok; ошибок связей нет

Проверка сохранности включает повторное открытие файловой базы другим процессом и отдельный HTTP-прогон с перезапуском на этапе разработки API. В приложении А показаны общий список (рисунок А.1), форма редактирования (рисунок А.2), ошибка цены (рисунок А.3) и запрет удаления занятой категории (рисунок А.4). Снимки фиксируют реальные страницы: часть относится к раннему сценарию изменения одного товара, часть — к большому набору. Все изображения относятся к учебному каталогу. Подход к документированию условий и происхождения результата рассматривается в работе Ирвинга и соавторов [4].

Проверки относятся к локальной работе с каталогом. Нагрузочное тестирование, одновременное редактирование несколькими менеджерами и перенос на PostgreSQL не проводились. Для многопользовательской

эксплуатации потребуются авторизация, журнал изменений, миграции схемы и проверка конкурирующих записей.

Проверки хранения и проверки API подтверждают разные части одного правила. Тест базы может выполнить прямую запись недопустимого значения и получить отказ ограничения SQLite. Тест API проверяет, что неправильный запрос вернул 422 или 409 и не изменил каталог. Если использовать только API, ограничение базы могло бы отсутствовать и оставаться незамеченным благодаря входной схеме. Если проверять только базу, могли бы сохраниться неверный код ответа и потеря введённых значений формы.

Граничные примеры выбираются по правилам конкретных полей. Для названия проверяются пустая строка, пробелы и превышение допустимой длины; для цены — ноль, лишняя точность, запрещённые представления и предел целого SQLite. Для количества отдельно рассматриваются отрицательное, дробное, логическое и слишком большое значения. Поиск проверяется на русском регистре и буквальных символах процента, подчёркивания и косой черты. Эти случаи связаны с реализацией схем и запроса, а не со случайным набором товаров.

Тест повторного артикула сравнивает запись до и после отклонённого обновления. Далее выполняется допустимое обновление, чтобы проверить возможность продолжить работу. При проверке переноса товара меняется category_id, после чего читаются идентификатор и название категории. Прежнюю категорию затем можно удалить, если товаров в ней больше нет. Такая последовательность проверяет связь объектов при изменении, а не только создание двух независимых записей с подходящими полями.

Для загрузчика искусственный сбой вызывается при выполнении INSERT товара. После исключения проверяется отсутствие и товаров, и категорий, добавленных неудачной транзакцией. Отдельный тест подменяет создание резервной копии ошибкой и подтверждает, что загрузка не начинается. В WAL-сценарии запись сначала фиксируется в журнале открытой базы, затем читается

из резервной копии. Так проверяются разные отказы: откат незавершённой загрузки, запрет записи без копии и включение уже зафиксированных данных.

Повторная загрузка оценивается не только по общему числу строк. В отдельном тесте до запуска создаются собственный товар и категория с пользовательским описанием. После первой загрузки изменяются цена и название одной демонстрационной позиции. Второй запуск должен сохранить все эти значения, пропустить существующие артикулы и не создавать новую копию товара. Проверка числа записей без сравнения содержимого не обнаружила бы ошибку, при которой загрузчик молча возвращает изменённые поля к исходным значениям.

Совпадение пользовательского артикула с артикулом набора также считается занятой позицией. Тест заранее создаёт товар DEMO-01-01-01 в другой категории и проверяет добавление только 1199 новых товаров при одном пропуске. Исходная запись сохраняет название и принадлежность к категории. Это сознательное правило сохранения пользовательских данных: загрузчик не определяет происхождение записи по названию. Для повторной демонстрации оно полезнее принудительного восстановления полного шаблона, которое могло бы уничтожить исправления менеджера.

После запуска отдельно проверяется готовность обычного сервера командой scripts.check_ready. Проверка открывает главную страницу с перенаправлением, списки и формы, получает локальные CSS и JavaScript, читает health и списки API. Затем по OpenAPI проверяется наличие десяти операций каталога. Ожидание ответа 200 дополняется проверкой содержания: заголовка HTML, формата JSON и необходимых полей списка. Поэтому случайно запущенный на том же порту другой сервер не удовлетворит всем условиям только благодаря доступности адреса.

Структура рабочей базы проверяется без добавления тестовых товаров. Через SQLAlchemy читаются имена таблиц, столбцы товара и описание внешнего ключа с запретом удаления категории. Для соединения проверяется включение PRAGMA foreign_keys. Такой контроль дополняет отдельный ответ

health, который возвращает фиксированный статус и сам не выполняет запрос к каталогу. Разделение позволяет обнаружить ситуацию, когда HTTP-процесс уже запущен, но требуемая схема или параметры соединения не соответствуют приложению. Команда не изменяет рабочие записи, поэтому её можно выполнять после обычного запуска и обновления исходников.

Повторение тестов относится к той версии кода и зависимостей, на которой выполнен запуск. Успешный результат не переносится автоматически на изменённую схему или другой сервер базы. Поэтому в документации разделены pytest, HTTP-прогон с перезапуском и браузерные отчёты с датой и условиями. При изменении надписей старые снимки остаются результатами прежнего прогона. Для проверки новых пользовательских сценариев нужны соответствующие новые действия, а не переименование сохранённых файлов отчёта.

ЗАКЛЮЧЕНИЕ

Разработан локальный модуль управления каталогом товаров на Python 3.12, FastAPI и SQLite. Через веб-интерфейс и JSON API можно создавать, просматривать, изменять и удалять товары и категории, искать позиции и применять фильтры. Для демонстрации используется ассортимент канцелярских товаров.

Для реализации выбран самостоятельный модуль, поскольку задание требует разработки и объяснения собственного кода. SQLite позволяет запускать его без отдельного сервера базы данных и создавать временные базы для тестов. Документация PostgreSQL использована при сравнении решений; переноса приложения на эту СУБД не было.

Спроектированы две основные сущности и связь один ко многим. Категория хранится отдельно от товаров, а её идентификатор используется как внешний ключ. Цена представлена целыми копейками, остаток — неотрицательным целым числом. Нормализованное название используется для поиска без учёта регистра и проверки имени категории. Разделены входные схемы, хранимые модели, общие операции и представления. Это позволило использовать одни правила в HTML и JSON API.

Реализованы десять операций API, карточки и формы, подтверждение удаления, обработка ошибок с сохранением ввода. Поиск сочетается с категорией, включительными ценовыми границами и наличием. Фиксированный перечень сортировок и дополнительный порядок по идентификатору делают выборку однозначной. Пагинация ограничивает страницу десятью записями и сохраняет условия при переходах. Основные формы работают без JavaScript, а локальное отображение дат дополняется скриптом.

Для демонстрации подготовлены 1200 товаров в 12 категориях. Перед загрузкой в существующую базу создаётся резервная копия, затем недостающие записи добавляются одной транзакцией. Повторный запуск

пропускает занятые артикулы и сохраняет пользовательские изменения. Проверки целостности SQLite нарушений не выявили.

Пройдены 154 автоматических теста, 14 основных браузерных сценариев и 6 дополнительных сценариев большого каталога. При браузерной проверке найдены и исправлены расширение страницы на узком экране и выравнивание таблицы. Условия запуска и отчёты сохранены в документации проекта. Эти результаты подтверждают проверенные сценарии, но не исключают ошибок за их пределами.

Модуль выполняет предусмотренные заданием операции с каталогом в локальной среде. Перед использованием несколькими менеджерами нужно добавить авторизацию и историю изменений, подготовить миграции и проверить одновременную запись. Для учебной демонстрации подготовлены исходники, инструкция запуска, демонстрационные данные и материалы защиты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание.

2. Московский финансово-юридический университет МФЮА. Методические указания по подготовке и оформлению курсовых работ/проектов. — Москва, 2025. — Текст : электронный.

3. Downey, A. B. Think Python: How to Think Like a Computer Scientist / A. B. Downey. — 3rd ed. — O’Reilly Media, 2024. — ISBN 978-1-098-15543-8. — URL: https://github.com/AllenDowney/ThinkPython/tree/v3 (дата обращения: 21.10.2026). — Текст : электронный.

4. Research Software Engineering with Python: Building Software that Makes Research Possible / D. Irving, K. Hertweck, L. Johnston [et al.]. — Chapman & Hall/CRC Press, 2021. — ISBN 978-0-367-69834-8. — URL: https://github.com/merely-useful/py-rse (дата обращения: 21.10.2026). — Текст : электронный.

5. Percival, H. Architecture Patterns with Python / H. Percival, B. Gregory. — O’Reilly Media, 2020. — ISBN 978-1-492-05220-3. — URL: https://github.com/cosmicpython/book (дата обращения: 21.10.2026). — Текст : электронный.

6. Percival, H. J. W. Test-Driven Development with Python / H. J. W. Percival. — 3rd ed. — O’Reilly Media, 2025. — ISBN 978-1-098-14871-3. — URL: https://github.com/hjwp/Book-TDD-Web-Dev-Python (дата обращения: 21.10.2026). — Текст : электронный.

7. Wilson, G. Software Design by Example: A Tool-Based Introduction with Python / G. Wilson. — URL: https://github.com/gvwilson/sdxpy (дата обращения: 21.10.2026). — Текст : электронный.

8. Winters, T. Software Engineering at Google: Lessons Learned from Programming Over Time / T. Winters, T. Manshreck, H. Wright. — O’Reilly Media, 2020. — ISBN 978-1-492-08279-8. — URL: https://github.com/abseil/abseil.github.io/blob/master/resources/swe-book/html/ch11.html (дата обращения: 21.10.2026). — Текст : электронный.

9. Zeller, A. The Debugging Book / A. Zeller. — 2021. — URL: https://github.com/uds-se/debuggingbook (дата обращения: 21.10.2026). — Текст : электронный.

10. The Fuzzing Book / A. Zeller, R. Gopinath, M. Böhme [et al.]. — 2021. — URL: https://github.com/uds-se/fuzzingbook (дата обращения: 21.10.2026). — Текст : электронный.

11. Python Software Foundation. Python 3.12 Documentation: sqlite3, decimal, unicodedata. — URL: https://github.com/python/cpython/tree/3.12/Doc/library (дата обращения: 21.10.2026). — Текст : электронный.

12. PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Data Definition. — URL: https://github.com/postgres/postgres/blob/REL_18_STABLE/doc/src/sgml/ddl.sgml (дата обращения: 21.10.2026). — Текст : электронный.

13. FastAPI. Templates: официальная документация. — URL: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/advanced/templates.md (дата обращения: 21.10.2026). — Текст : электронный.

14. SQLAlchemy. Session Basics: официальная документация. — URL: https://github.com/sqlalchemy/sqlalchemy/blob/main/doc/build/orm/session_basics.rst (дата обращения: 21.10.2026). — Текст : электронный.

15. Jinja. Template Designer Documentation. — URL: https://github.com/pallets/jinja/blob/main/docs/templates.rst (дата обращения: 21.10.2026). — Текст : электронный.

16. Pydantic. Strict Mode: официальная документация. — URL: https://github.com/pydantic/pydantic/blob/main/docs/concepts/strict_mode.md (дата обращения: 21.10.2026). — Текст : электронный.

17. pytest. How to Use Fixtures: официальная документация. — URL: https://github.com/pytest-dev/pytest/blob/main/doc/en/how-to/fixtures.rst (дата обращения: 21.10.2026). — Текст : электронный.

18. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — 2022. — RFC 9110. — URL: https://github.com/httpwg/http-core/blob/main/rfc9110.xml (дата обращения: 21.10.2026). — Текст : электронный.

19. Microsoft. Playwright for Python: Getting Started — Library. — URL: https://github.com/microsoft/playwright/blob/main/docs/src/library-python.md (дата обращения: 21.10.2026). — Текст : электронный.

20. Uvicorn. Settings: официальная документация. — URL: https://github.com/encode/uvicorn/blob/main/docs/settings.md (дата обращения: 21.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ

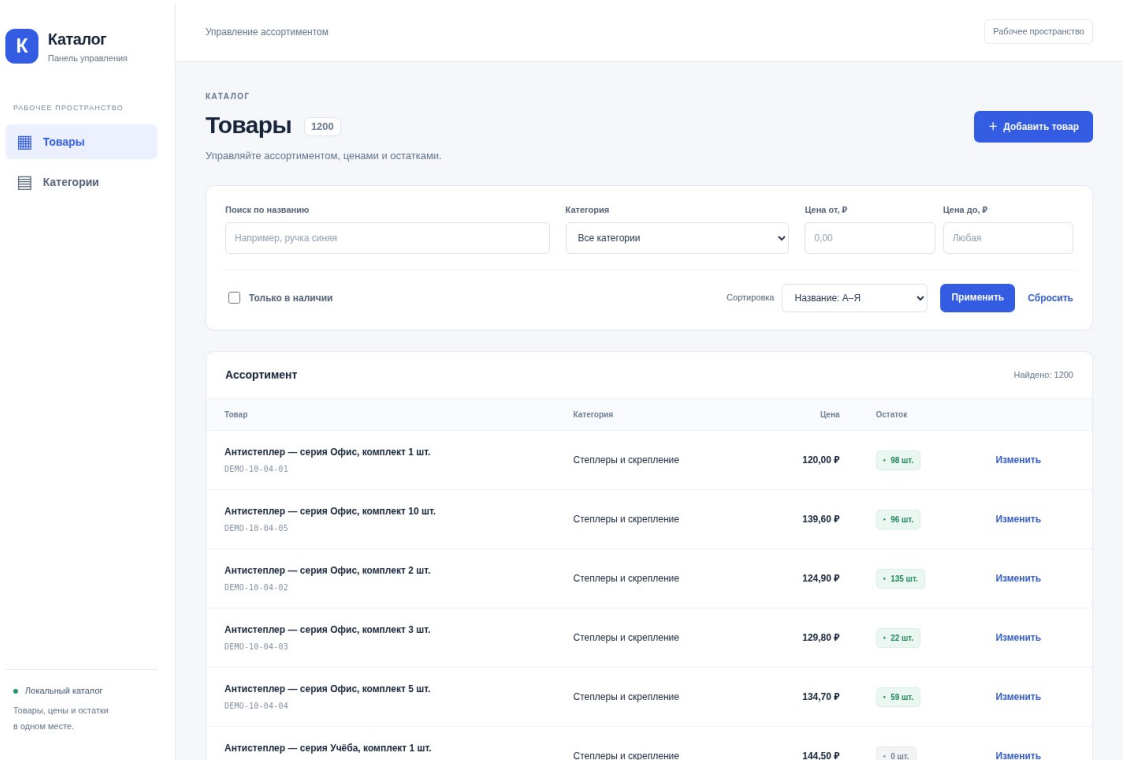


Рисунок А.1 — Каталог из 1200 демонстрационных товаров

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

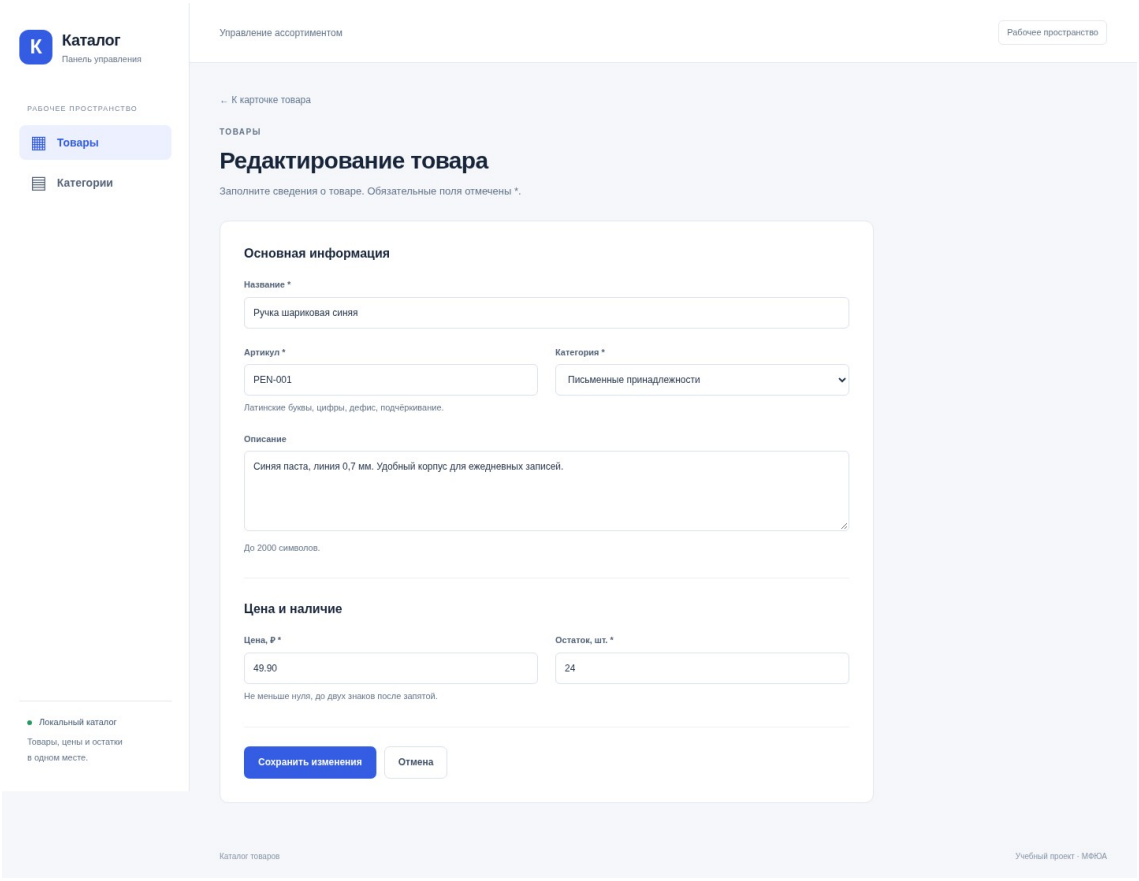


Рисунок А.2 — Форма редактирования товара

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

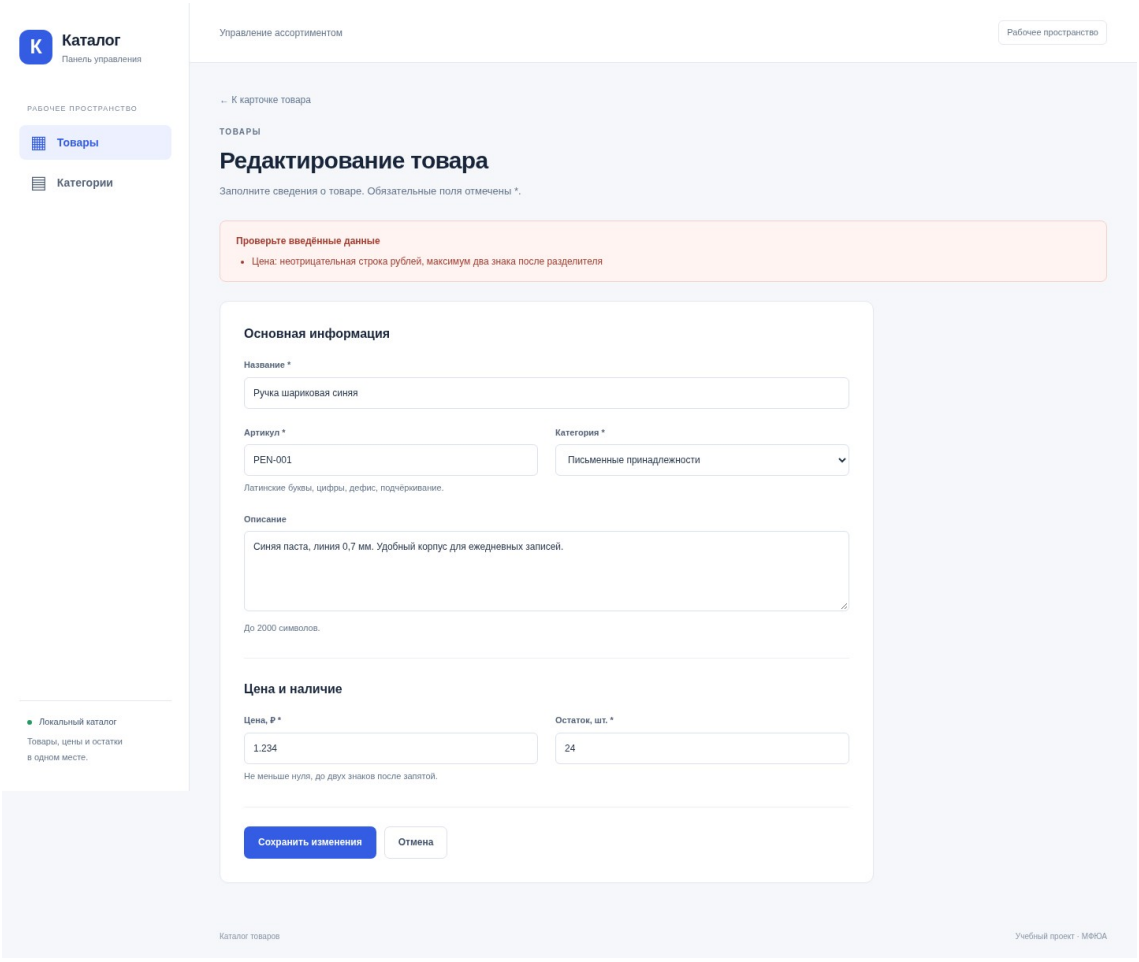


Рисунок А.3 — Сохранение ввода при неправильной цене

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

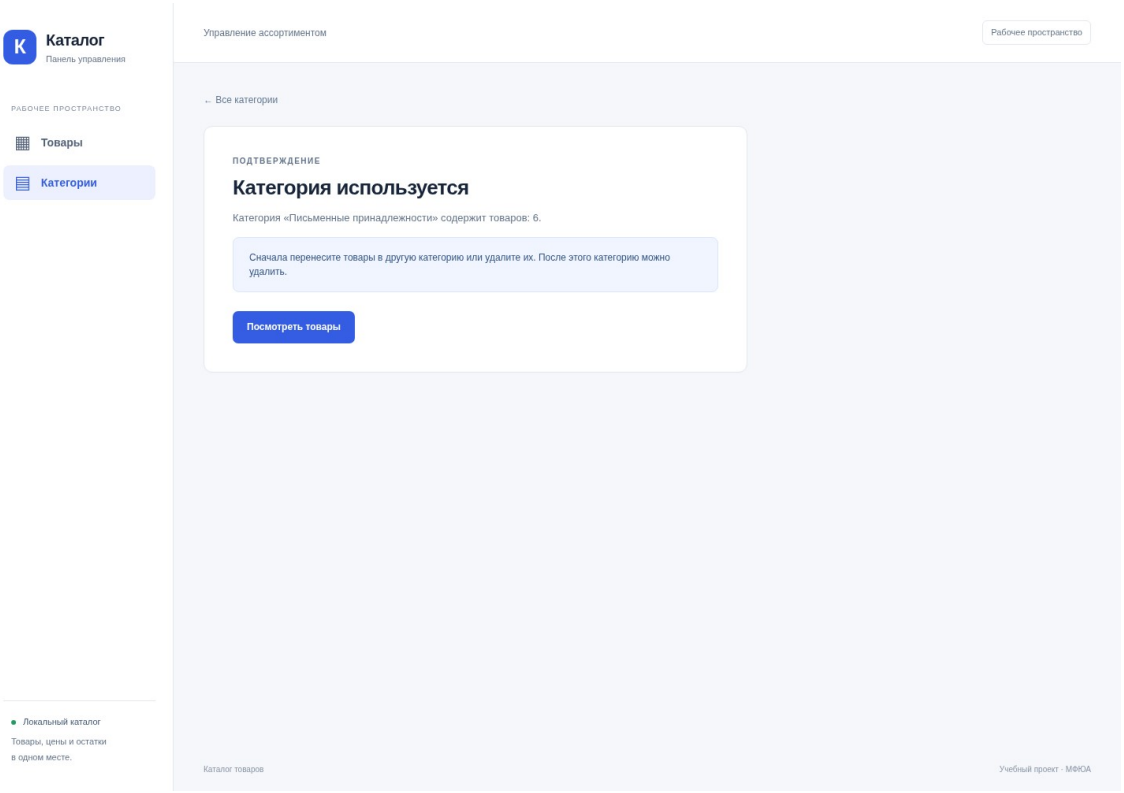


Рисунок А.4 — Запрет удаления категории с товарами

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.